

API Manual

Reference of the HTTP routes of the Data Management Portal

Generated on: 21/07/2026

Base: /api

Authentication: JWT HS256 (Bearer)

1. Fundamentals

Every route lives under the /api prefix, resolved by the front controller index.php. The portal works in a subfolder: if it sits at http://server/data, the base is http://server/data/api.

Requests and responses use JSON. Errors always follow the same shape, with the message in erro:

```
{ "erro": "Usuario ou senha invalidos." }
```

Authentication

Login returns a JWT token that must accompany every other call:

```
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

The token carries exp and iat. The issue time enables mass revocation: on a password change the server records a cutoff and rejects any token issued before it. Logout revokes only the token in use, by its jti.

Response codes

Code	Meaning
200	Success, with a JSON body.
201	Record created.
204	Success with no body (deletions).
400	Malformed request or invalid operation.
401	Missing, invalid or expired token.
403	Authenticated, but not allowed to perform the action.
404	Route or record not found.
422	Data rejected during validation.

Code	Meaning
429	Attempt or resend limit reached.

Roles

Role	Can do
leitura	Read the modules allowed for the account.
escrita	Read and maintain records in the allowed modules.
admin	Everything escrita can, plus users, lists and configuration.
master	Everything admin can, plus the audit trail and actions on another master.

There is no single authorization gate: each route explicitly calls `exigirLogin()`, `exigirAdmin()` or `exigirMaster()`. When adding a new route, that call is mandatory -- without it the route is public.

2. Session and account

POST /auth/login

Access: public

Authenticates and returns the token. If the account uses two-step verification it returns no token: it e-mails a code and responds with a temporary verification token.

Request

```
{
  "usuario": "admin",
  "senha": "SenhaForte1"
}
```

Response

```
{
  "token": "eyJ0eXAiOiJKV1Qi...",
  "usuario": {
    "id": 1,
    "username": "admin",
    "role": "master"
  }
}
```

Failures are limited to 5 per user+IP and 20 per IP, with a temporary block. The response does not say whether the login or the password was wrong, so valid accounts are not revealed.

POST /auth/mfa/verificar

Access: public

Checks the six-digit code and completes the login.

Request

```
{
  "token": "a1b2c3...",
  "codigo": "482915"
}
```

Response

```
{
  "token": "eyJ0eXAiOiJKV1Qi...",
  "usuario": { "id": 1, "username": "admin" }
}
```

The code is valid for 10 minutes and allows at most 5 checks. It is stored only as an HMAC of "token|code".

POST /auth/mfa/reenviar

Access: public

Resends the code. Minimum interval of 60 seconds and a cap of 5 resends per login session.

Request

```
{ "token": "a1b2c3..." }
```

Response

```
{ "ok": true }
```

GET /auth/me

Access: any authenticated account

Data of the account in use, including whether it has a photo.

Response

```
{
  "id": 1,
  "username": "admin",
  "nome_completo": "Administrador",
  "role": "master",
  "modulos_permitidos": "acessos,jobs",
  "mfa_ativo": 0,
  "tem_foto": true
}
```

PUT /auth/me

Access: any authenticated account

Updates name, e-mail and visual preferences of the own account.

Request

```
{
  "nome_completo": "Administrador",
  "email": "admin@empresa.com",
  "cor_tema": "padrao"
}
```

Response

```
{ "ok": true }
```

POST /auth/change-password

Access: any authenticated account

Changes the password. Invalidates all previous tokens, including on other devices.

Request

```
{
  "senha_atual": "SenhaAntiga1",
  "senha_nova": "SenhaForte2"
}
```

Response

```
{ "ok": true }
```

POST /auth/logout

Access: any authenticated account

Revokes the token in use.

Response

```
204 no body
```

3. Users

GET /usuarios

Access: admin

Lists the accounts. The password is never returned.

Response

```
[
  {
    "id": 1,
    "username": "admin",
    "role": "master",
    "ativo": 1,
    "tem_foto": true
  }
]
```

POST /usuarios

Access: admin

Creates an account.

Request

```
{
  "username": "maria",
  "nome_completo": "Maria Silva",
  "email": "maria@empresa.com",
  "senha": "SenhaForte1",
  "role": "escrita",
  "modulos_permitidos": "acessos,jobs"
}
```

Response

```
{ "id": 7 }
```

PUT /usuarios/{id}

Access: admin

Updates the account. Sending a blank password keeps the current one.

Request

```
{
  "nome_completo": "Maria S. Silva",
  "role": "admin",
  "ativo": 1
}
```

Response

```
{ "ok": true }
```

Only master can grant or remove the master role.

DELETE /usuarios/{id}

Access: admin

Removes the account and its photo. You cannot remove yourself, and only master removes another master.

Response

```
204 no body
```

PATCH /usuarios/{id}/ativo

Access: admin

Enables or disables the account without erasing history.

Request

```
{ "ativo": 0 }
```

Response

```
{ "ok": true }
```

PUT /usuarios/{id}/permissoes

Access: admin

Sets role and allowed modules.

Request

```
{
  "role": "escrita",
  "modulos_permitidos": "acessos, backup, jobs"
}
```

Response

```
{ "ok": true }
```

User photo

The photo lives in the database, in its own table, and travels as a data URI inside the JSON -- not as a file. That is because the tag cannot send the Authorization header, so the image must be fetched through the same authenticated call as everything else.

GET /usuarios/{id}/foto

Access: the owner or an admin

Returns the photo. Responds 404 when the account has no photo.

Response

```
{ "foto": "..." }
```

PUT /usuarios/{id}/foto

Access: the owner or an admin

Sets the photo. Accepts PNG, JPEG, WEBP and GIF as a data URI, up to about 1 MB.

Request

```
{ "foto": "..." }
```

Response

```
{ "ok": true }
```

SVG is rejected, and the content must really be an image -- the header alone is not enough. Only master can change a master photo.

DELETE /usuarios/{id}/foto

Access: the owner or an admin

Removes the photo, falling back to the name initial.

Response

```
204 no body
```

4. Governance modules

The modules share the same set of routes, only the name in the path changes. The available modules are: acessos, mudancas, backup, restore, dicionario, integracoes, jobs.

Every call requires login and goes through per-module access control: an account only sees the modules allowed in its record.

Route	Does
GET /{modulo}	Lists, with search and pagination.
POST /{modulo}	Creates a record.
PUT /{modulo}/{id}	Updates a record.
DELETE /{modulo}/{id}	Removes a record.
DELETE /{modulo}/lote	Removes several records at once.
GET /{modulo}/exportar	Exports the module as JSON.

GET /jobs?busca=carga&pagina;=1

Access: account with access to the module

Listing example. The busca parameter scans the text columns of the module.

Response

```
[
  {
    "id": 3,
    "nome": "Carga diaria de vendas",
    "frequencia": "Diario",
    "comando": "[{\"passo\":\"Extrair\",\"executa\":\"sp_extrai_vendas\"}]",
    "agendamentos": "[{\"inicio\":\"02:00\",\"fim\":\"03:30\"}]",
    "status": "Ativo",
    "criado_por": "admin"
  }
]
```

In Jobs, the comando and agendamentos fields hold JSON lists. The portal handles reading and writing; anyone consuming the API directly must treat those two fields as JSON text.

POST /acessos

Access: account with write access to the module

Record creation. The criado_por field is set by the server and ignored if sent in the request.

Request

```
{
  "data": "2026-07-21",
  "usuario": "maria",
  "servidor": "SRV-BD-01",
  "objeto": "vendas",
  "nivel": "leitura",
  "status": "Ativo"
}
```

Response

```
{ "id": 42 }
```

5. Configuration and support

Route	Access	Does
GET /dashboard/stats	authenticated	Figures for the home screen.
GET /dashboard/export	authenticated	Exports the dashboard.
GET /tipos	authenticated	Editable Cadastro lists.
POST /tipos	admin	Creates a list option.
PUT /tipos/{id}	admin	Renames an option.
DELETE /tipos/{id}	admin	Removes an option.
GET /config/projeto	authenticated	Name, logo and session timeout.
PUT /config/projeto	admin	Changes the portal identity.
GET /config/email	admin	SMTP settings (without the password).
PUT /config/email	admin	Saves the SMTP settings.
POST /config/email/testar	admin	Sends a test e-mail.
GET /seguranca/stats	admin	Recent blocks and attempts.
GET /auditoria	master	Audit trail.
GET /sistema/info	authenticated	Version and database engine in use.
GET /documentacao	authenticated	User manual as PDF.
GET /documentacao/api	authenticated	This manual, as PDF.
GET /documentacao/diagrama	authenticated	Data dictionary as PDF.

The three documentation routes have an English version with the `-en` suffix: `/documentacao/manual-en`, `/documentacao/api-en` and `/documentacao/diagrama-en`.

6. Full example

Login and reading a module, with curl:

```
# 1) login
curl -s -X POST http://servidor/dados/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"usuario":"admin","senha":"SenhaForte1"}'

# 2) usa o token devolvido
curl -s http://servidor/dados/api/jobs \
  -H "Authorization: Bearer $TOKEN"
```